

Algoritmo para reducir la cantidad de puntos en un relieve poliedral*

Simón Díaz Monroy^a

^a*simondiaz@javeriana.edu.co*

Abstract

Algoritmo para simplificar relieves poliedrales usando triangulaciones de Delaunay. Se eliminan vértices cuya altura se desvía de un umbral adaptativo basado en media y desviación estándar del vecindario. Las pruebas muestran reducciones superiores al 80 %.

Keywords: Triangulación de Delaunay, reducción de puntos, umbral

1. Formulación de corrección del algoritmo

El algoritmo actual presenta un comportamiento no deseado: elimina puntos en zonas de alto detalle mientras conserva una alta densidad en regiones planas, donde la información es redundante. El efecto esperado es el opuesto: reducir la concentración de puntos en áreas homogéneas y preservar aquellos que capturan variaciones significativas del relieve.

Se propone modificar el criterio de eliminación con las siguientes reglas:

1. **Conservar vecinos** Si un vértice satisface el criterio de permanencia ($|v_z - \mu_z| < \Upsilon$), sus vecinos directos también se conservan, por considerarse parte de la misma región representativa.
2. **Propagar eliminación.** Si un vértice no cumple el criterio ($|v_z - \mu_z| \geq \Upsilon$), se conserva y se eliminan todos sus vecinos directos, reteniendo únicamente un representante de la zona.

De esta manera, cada región del relieve conserva al menos un punto representativo mientras se eliminan los vecinos redundantes.

2. Análisis del problema

A partir de un umbral de altura (elevación positiva en el eje z) denotado por Υ , se toma cada vértice de una triangulación de Delaunay y se eliminan

*Describe un algoritmo que reduce la cantidad de puntos en un relieve poliedral a partir de un umbral

Email address: *simondiaz@javeriana.edu.co* (Simón Díaz Monroy)

aquellos cuyo valor de altura v_z sea mayor o igual a dicho umbral. El umbral se calcula en función de dos parámetros: el tamaño del vecindario δ , que determina la cantidad de vecinos considerados alrededor de cada vértice, y un número real positivo γ , que actúa como factor de escala para controlar qué tan permisivo es el criterio de eliminación.

Otro aspecto fundamental del problema es que, tras realizar las eliminaciones necesarias, se debe reconstruir la triangulación de Delaunay, ya que la solución propuesta tiene como objetivo reducir la cantidad de datos requeridos para representar el terreno dentro del margen definido por el umbral.

2.1. Formalización

Dado un grafo conexo $G = (V, E)$ que cumple con las propiedades de una triangulación de Delaunay, un parámetro $\delta \in \mathbb{N} \setminus \{0\}$ y un parámetro $\gamma \in \mathbb{R}^+$, para cada vértice v se define su vecindario $V_\delta(v) = \{v^0, v^1, \dots, v^\delta\}$ y se eliminan todos aquellos vértices que satisfacen la condición:

$$|v_z - \mu_z| \geq \gamma \cdot \sigma_z \quad (1)$$

Donde:

1. v_z es la coordenada en el eje z del vértice v .
2. μ_z es el promedio de las alturas del vecindario $V_\delta(v)$.
3. σ_z es la desviación estándar de las alturas del vecindario $V_\delta(v)$.
4. $\gamma \cdot \sigma_z$ define el umbral Υ .

Una vez realizadas las eliminaciones, se debe reconstruir la triangulación de Delaunay.

2.1.1. Predicados

Para abordar el problema es necesario disponer de una operación que, dado un vértice v de la triangulación, retorne sus vecinos, de modo que pueda aplicarse iterativamente para construir el vecindario $V_\delta(v)$:

$$\text{neigh}(v, \delta) = \{v^0, v^1, \dots, v^\delta\} \quad (2)$$

3. Diseño del problema

3.1. Entradas

- $G = (V, E)$, grafo que representa la triangulación de Delaunay
 - $V = \{v_i \in \mathbb{R}^3 \mid 0 \leq i \leq n\}$, conjunto de n puntos
 - $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$, conjunto de aristas no dirigidas
- $\gamma \in \mathbb{R}^+$, factor de escala de Υ
- $\delta \in \mathbb{N} \setminus \{0\}$, tamaño del vecindario

3.2. Salidas

- $G' = (V', E')$, triangulación donde $|V'| < |V|$

4. Algoritmo de solución

4.1. Estructuras de datos

- M , secuencia de booleanos que marcan si se ha pasado por un vértice
- Q , cola de vértices con su profundidad
- V_δ , vecindario de vértices para un numero de saltos δ

4.2. Descripción

La estrategia para eliminar los puntos por debajo del umbral Υ es recorrer los vértices de la triangulación y por cada vértice v :

1. Obtener su vecindario usando el predicado $\text{neigh}(v, \delta)$
2. Calcular el umbral Υ local para el vértice elegido
3. Eliminar los vértices que sean mayores o iguales a Υ
4. Recalcular la triangulación con los puntos restantes

Para el predicado $\text{neigh}(v, \delta)$ se define un procedimiento cuyos pasos son:

1. Marcar el vértice v en M y colocarlo Q
2. Recorrer en sentido antihorario los vértices a los que esta directamente conectados a v :
 - a) Marcar cada vértice en M y introducirlo en Q registrando su profundidad con respecto a δ
 - b) Si había sido marcado previamente o la profundidad alcanzada es igual a δ no se introduce en Q .
3. Sacar el vértice v de Q y insertarlo en V_δ
4. Repetir los pasos 1, 2 y 3 con el siguiente vértice de la cola hasta que la cola este vacía

Algorithm 1 Reducción de puntos de un relieve poliedral

Require: $G = (V, E) \mid |V| > 0 \wedge |E| > 0$ **Require:** $\gamma \in \mathbb{R}^+$ **Require:** $\delta \in \mathbb{N} \setminus \{0\}$

```

1: procedure REDUCEOVERTHRESHOLD( $G, \gamma, \delta$ )
2:   for  $v$  in  $vertices(G)$  do
3:      $V_\delta \leftarrow neig(v, \delta)$ 
4:      $\mu_z \leftarrow \text{AVGHEIGHT}(V_\delta)$ 
5:      $\sigma_z \leftarrow \text{SIGMAHEIGHT}(V_\delta)$ 
6:      $\Upsilon \leftarrow \gamma \cdot \sigma_z$ 
7:     for  $v'$  in  $V_\delta$  do
8:       if  $|v'_z - \mu_z| \geq \Upsilon$  then
9:          $\text{ERASE}(G, v')$ 
10:      end if
11:    end for
12:     $G' \leftarrow \text{TRIANGULATE}(G)$ 
13:  end for
14:  return  $G'$ 
15: end procedure

```

Algorithm 2 Obtener vecindario δ

Require: $v \in \mathbb{R}^2$ **Require:** $\delta \in \mathbb{N} \setminus \{0\}$

```

1: procedure  $neig(v, \delta)$ 
2:    $M, V_\delta \leftarrow \emptyset$ 
3:    $Q \leftarrow \text{CREATEQUEUE}()$ 
4:    $M_v \leftarrow \text{True}$ 
5:    $\text{PUSH}(Q, v)$ 
6:    $d \leftarrow 0$  ▷ Profundidad
7:   while  $Q$  is not empty do
8:      $v_q \leftarrow \text{DEQUEUE}(Q)$ 
9:      $V_\delta \cup v_q$ 
10:     $V_v \leftarrow \text{ITERCCLOCKWISE}(v_q)$ 
11:     $d++$ 
12:    for  $v'$  in  $V_v$  do
13:      if  $M_{v'}$  is false  $\wedge d \leq \delta$  then
14:         $M_{v'} \leftarrow \text{True}$ 
15:         $\text{PUSH}(Q, v')$ 
16:      end if
17:    end for
18:  end while
19:  return  $V_\delta$ 
20: end procedure

```

4.3. Análisis de complejidad

En el peor caso, para cada uno de los n vértices, el predicado $\text{neigh}(v, \delta)$ recorre todo el grafo ($O(n)$) y la reconstrucción mediante $\text{triangulate}(G)$ cuesta $O(n \log n)$. El bucle externo repite estos pasos n veces, lo que resulta en una complejidad total de

$$O(n^2 \log n) \quad (3)$$

4.4. Notas de implementación

La implementación utiliza **CGAL** para las operaciones geométricas sobre la triangulación de Delaunay. Los aspectos técnicos principales son:

- **Triangulación con altura.** Se usa `Triangulation_vertex_base_with_info_2` para almacenar la coordenada z en el campo `info()` de cada vértice, con un kernel de predicados exactos y construcciones inexactas (`Exact_predicates_inexact_constructions_kernel`).
- **Vecindario.** La función `neigh()` implementa un BFS sobre la triangulación usando `Vertex_circulator` (devuelto por `incident_vertices()`) para recorrer los vecinos en orden antihorario, limitado a profundidad δ .
- **Eliminación.** Los vértices que superan Υ se remueven con `remove(Vertex_handle)`, que reconstruye la triangulación local preservando la propiedad de Delaunay.
- **Mapeo.** Un arreglo `origin_points` mantiene la correspondencia punto-índice original para evitar invalidaciones por eliminación.
- **Propagación de vecindario:** Al evaluar Υ para un vértice v , se marcan para eliminación todos los vértices de $V_\delta(v)$ que lo superen. Dado que los vecindarios se solapan, un mismo vértice puede marcarse desde distintos centros.
- **Inestabilidad numérica en Υ :** Cuando σ_z es cercano a cero, errores de punto flotante en el cálculo de μ_z y σ_z producen un umbral $\Upsilon = \gamma \cdot \sigma_z$ degenerado, eliminando vértices que deberían conservarse.

5. Resultados

Archivo	δ	γ	Puntos iniciales	Marcados	Eliminados	Restantes
heightmap-70x70.png	2	1.0	4900	15644	4400	500
tamriel-50x41.png	2	1.5	2050	1361	545	1505

Cuadro 1: Estadísticas de reducción para dos conjuntos de prueba

Con el algoritmo corregido se alcanzó una reducción del 89.8 % en la primera prueba y del 26.6 % en la segunda. La diferencia se debe al mayor γ en el segundo caso, que hace el umbral más abierto, y al nuevo criterio de conservación por supervivencia que retiene más puntos en zonas representativas.